

**SIMATS ENGINEERING
ASSESSMENT TOOL**

COURSE NAME: Theory of Computation COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO1: *Analyze fundamental mathematical concepts of automata theory for formal languages and decision problems(BL:3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:25 Duration : 1 Hour

Simulation

Code of Conduct:

I _____U.Divya/192372277_____ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Divya

SIMATS ENGINEERING ASSESSMENT TOOL

- 1 Design a **Non-Deterministic Finite Automaton (NBA)** for an **Automatic Door System** based on sensor inputs. An automatic door system can be modeled using an NBA to represent different states and transitions. The system consists of four states: *Closed*, *Opening*, *Open*, and *Closing*. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the *Closed* state. When a person is detected (P), the system may non-deterministically transition from *Closed* to *Opening*. Once opened, the system moves to the *Open* state. If no person is detected (N), the door transitions to *Closing* and eventually returns to the *Closed* state. Non determinism occurs when the system may remain in the *Open* state even if a person is detected or may start closing if no detection occurs.

States

- **q0** – Closed (Start State)
- **q1** – Opening
- **q2** – Open
- **q3** – Closing

Input Alphabet

$$\Sigma = \{P, N\}$$

- **P** = Person Detected
- **N** = No Person Detected

Start State

q0

Final State

$$F = \{q_2\}$$

Transition Table

Current State	P	N
$\rightarrow q_0$	{q1}	{q0}
q1	{q2}	{q1}
*q2	{q2}	{q2,q3}
q3	{q1}	{q0}

Non-determinism:

From **q2 (Open)** on input **N**, the automaton can either:

- remain in **Open (q2)**, or
- move to **Closing (q3)**.

- 2 Design a finite automaton to model a traffic light control system at a road intersection. The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red $\rightarrow$ Green $\rightarrow$ Yellow $\rightarrow$ Red). Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.

States

- q0 – Red
- q1 – Green
- q2 – Yellow

Input

T = Timer Signal

Start State

q0

Transition Table

Current State	Timer(T)	Next State
→ q0 (Red)	T	q1(Green)
q1(Green)	T	q2(Yellow)
q2(Yellow)	T	q0(Red)

Timing Constraints

- Red = 30 seconds
- Green = 45 seconds
- Yellow = 5 seconds

Extensions

- **Pedestrian Signal:** Add a Walk state activated by a pedestrian button.
- **Emergency Vehicle:** Override current state and switch to Green for the emergency direction.
-

3 Design DFA using simulator to accept the string the end
with ab over set {a,b} W= aaabab

Given String:

W = aaabab

States

- q0 – Start
- q1 – Last symbol is 'a'

- q2 – Ends with "ab" (Accept)

Transition Table

Current State	a	b
→ q0	q1	q0
q1	q1	q2
*q2	q1	q0

Simulation for W = aaabab

Input	State
Start	q0
a	q1
a	q1
a	q1
b	q2
a	q1
b	q2

Result: ✓ Accepted

- 4 Design a Deterministic Finite Automaton (DFA) using a simulator to accept only the input strings “c”, “bc”, and “bcaaa” over the alphabet {a, b, c}. The automaton should begin from an initial state and transition through intermediate states based on the sequence of input symbols, ensuring that each valid string follows a unique path. For example, the string “c” should be accepted directly from the start state, while “bc” should pass through states representing “b” and then “c”. Similarly, “bcaaa” should be processed through a sequence of states that track each additional “a” after “bc”. Any deviation from these exact patterns should lead to a dead state, ensuring that no other strings are accepted. Clearly define the states, transition functions, start state, and accepting states, and implement the DFA in a simulator such as Autosim to verify its correctness.

Alphabet

$$\Sigma = \{a, b, c\}$$

States

- q0 – Start
- q1 – Read b
- q2 – Read bc (Accept)
- q3 – Read bca
- q4 – Read bcaa
- q5 – Read bcaaa (Accept)
- q6 – Dead State

Accepting States

$$F = \{q2, q5\}$$

Transition Table

State	a	b	c
→ q0	q6	q1	q2
q1	q6	q6	q2
*q2	q3	q6	q6
q3	q4	q6	q6
q4	q5	q6	q6
*q5	q6	q6	q6
q6	q6	q6	q6

Accepted Strings

- c
- bc
- bcaaa

All other strings are rejected.

- 5 Design a Deterministic Finite Automaton (DFA) using a simulator to accept all strings over the alphabet {a, b} that begin with either “a” or “b”. The automaton should start in an initial state and immediately transition to an accepting state upon reading the first input symbol, since any valid

string must start with either “a” or “b”. After the first symbol is processed, the DFA should remain in an accepting state regardless of the subsequent sequence of “a” and “b”, thereby allowing any combination of characters to follow. Additionally, include a dead state to handle invalid or empty inputs if required by the simulator. Clearly define the states, transition rules, start state, and accepting states, and verify the design using a simulator such as Autosim to ensure that all valid strings are accepted and invalid ones are rejected.

Alphabet

$$\Sigma = \{a,b\}$$

States

- q_0 – Start
- q_1 – Accept
- q_2 – Dead State (optional for invalid/empty input)

Accepting State

$$F = \{q_1\}$$

Transition Table

Current State	a	b
$\rightarrow q_0$	q_1	q_1
$*q_1$	q_1	q_1
q_2	q_2	q_2

Working

- The DFA starts in **q_0** .
- Reading the first symbol (**a** or **b**) moves to **q_1** , which is an accepting state.

- The automaton remains in **q1** for any subsequent input.
- If the simulator requires a dead state, **q2** can be used for invalid or empty inputs.

Examples

a-Accepted

b-Accepted

aa-Accepted

abb-Accepted

baba-Accepted

ϵ (empty string)-Accepted

SIMATS ENGINEERING ASSESSMENT TOOL

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand